

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – Pseudocode Writing Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5 – Naturalization (BL5, BL6)	Assessment:	Assessment Tool 2 – Pseudocode Writing Task
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	Manoj M	Reg. No.:	192511060
Section / Batch:	CSE	Date:	27.08.2026

Instructions to Students

- Answer the following question with proper pseudocode and logical explanation.
- Clearly specify the algorithmic approach and complexity class wherever applicable.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) for complexity analysis.
- Include state-space representation, recursion steps, and pruning conditions in backtracking problems.
- Neat presentation and step-by-step justification are mandatory.

Question Type	Focus Area	Questions
Pseudocode Writing Task	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – Pseudocode Writing Task

1. Algorithm Logic (Backtracking & Pruning)

- **Q:** How do you structure the **recursive function** (e.g., Backtrack(current_selection, remaining_elements))?
- **Q:** How do you select the **next element** to cover to optimize the search? (e.g., picking the element with the fewest covering subsets).
- **Q:** How do you implement **pruning** to eliminate overlapping selections?
 - *Hint:* If selecting a subset covers an element already covered, the branch should be terminated immediately.

Code of Conduct

I _____Manoj(192511060)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm Design	30%				
Use of Control Structures	20%				
Pseudocode Structure	10%				
Completeness	20%				
Total Marks (100):					

CODE:

Your Code

```
1 #include <stdio.h>
2 #include <stdbool.h>
3
4 #define MAX_ELEMENTS 20
5 #define MAX_SUBSETS 20
6
7 int num_elements, num_subsets;
8 int subsets[MAX_SUBSETS][MAX_ELEMENTS];
9 int subset_sizes[MAX_SUBSETS];
10
11 int solution[MAX_SUBSETS];
12 int solution_size = 0;
13 bool covered[MAX_ELEMENTS];
14
15 // Function to check if a subset overlaps with currently covered elements
16 bool is_valid(int subset_idx) {
17     for (int i = 0; i < subset_sizes[subset_idx]; i++) {
18         int elem = subsets[subset_idx][i];
19         if (covered[elem]) {
20             return false; // Pruning condition: Overlap detected
21         }
22     }
23     return true;
24 }
25
26 // Function to update element coverage state
27 void set_coverage(int subset_idx, bool status) {
28     for (int i = 0; i < subset_sizes[subset_idx]; i++) {
29         int elem = subsets[subset_idx][i];
30         covered[elem] = status;
31     }
32 }
```

OUTPUT:

Input

```
7 6
3 0 3 6
3 1 4 5
3 0 2 5
```

Output

```
Exact Cover Found:
Subset 1: 1 4 5
Subset 4: 2 3
Subset 5: 0 6
```

Algorithm:

Minimum Linear Arrangement problem:

1. Define the Problem: Arrange graph vertices in a linear order to minimize the maximum edge length.
2. Define Constraints: Clearly state the rules for:
 - Vertex ordering (each vertex gets a unique position).
 - Edge distance (calculating the distance between connected vertices).
3. Analyze Impact: Discuss how these constraints affect which arrangements are possible (feasible).
4. Develop a Solution: Create a backtracking algorithm to solve this.
5. Apply Pruning: Implement techniques to cut off unproductive search paths and reduce the search space.